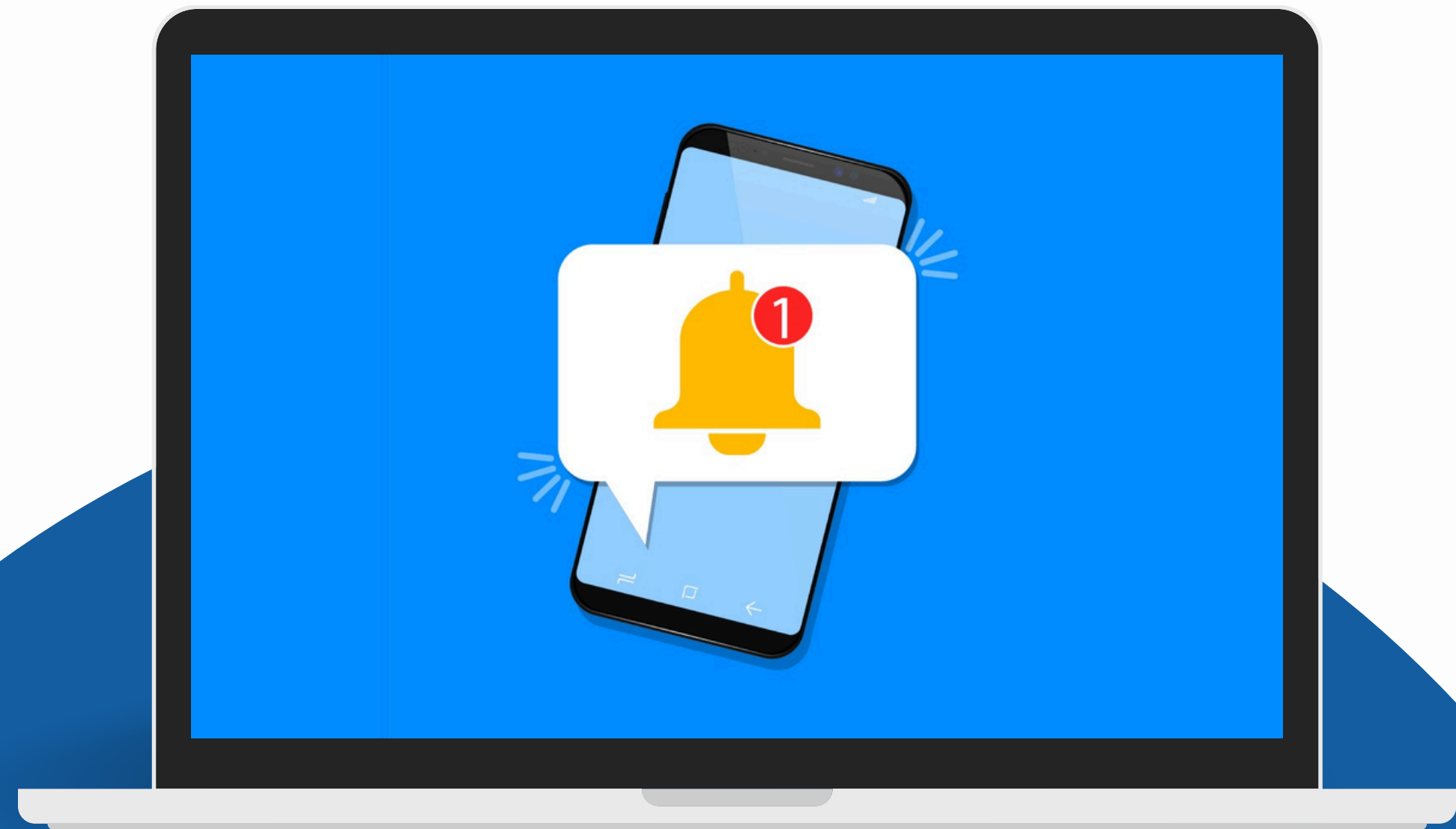


Notification Framework

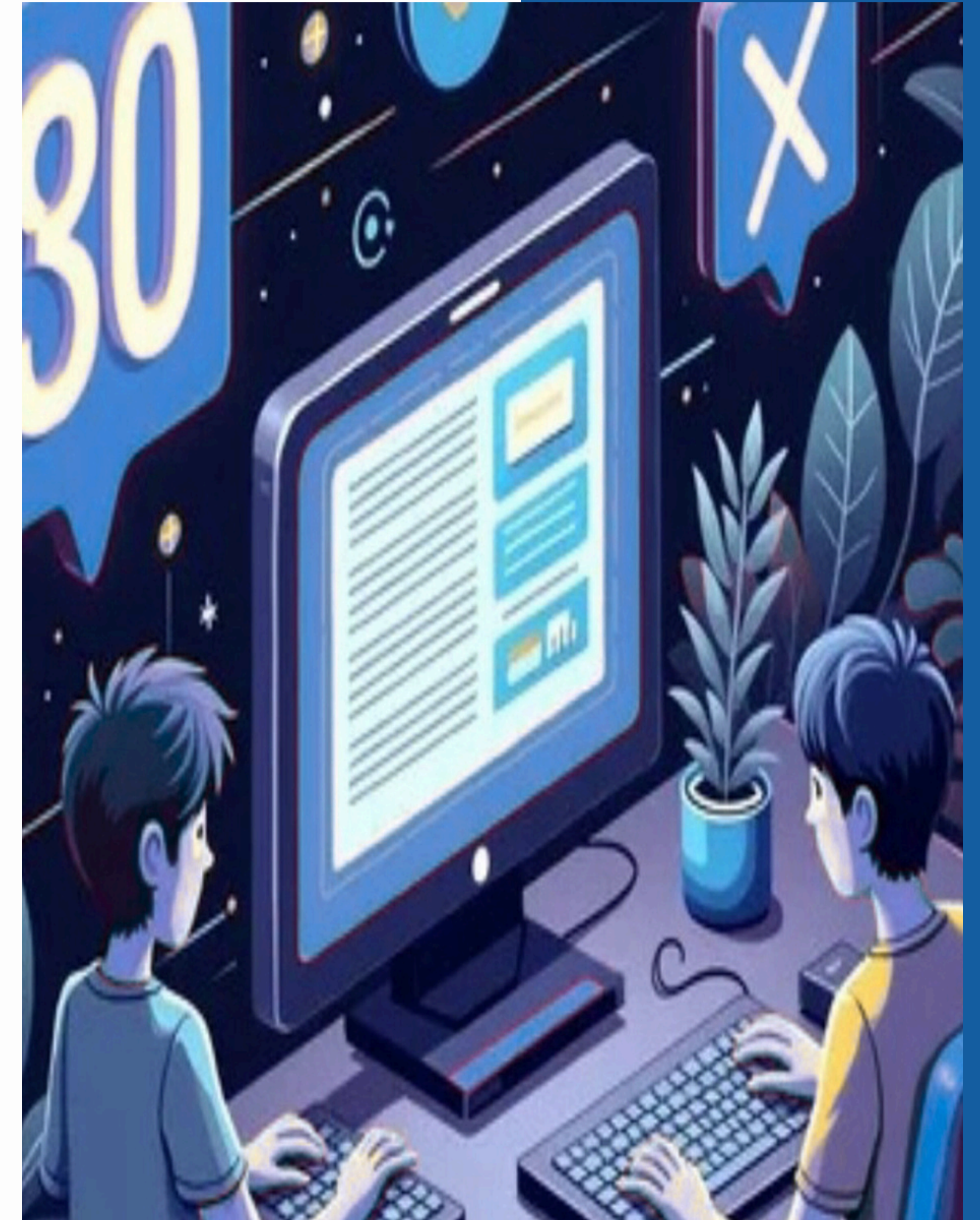
By: Group 18

**Vandana, Poojitha, Harshini,
Ruchika, Rashmika, Pranathi, Mahani**



Overview

▶ Title	01
▶ Background	03
▶ Problem Statement	04
▶ Our Understanding	05
▶ Architecture Diagram	06
▶ Objectives	07
▶ Vital Features	08
▶ Essential/Desired Feature	18
▶ Tech Stack	19
▶ Tech Stack Justification	20
▶ Sprint 1 Features	22



Background

As of early 2026, notification frameworks have evolved from simple “*send-and-forget*” push systems into **intelligent, multichannel supporting platforms**. The industry focus has shifted from raw delivery speed to cross-channel consistency, compliance, and AI-driven engagement optimization.

Eg: Native Infrastructure – Firebase Cloud Messaging (FCM), Apple Push Notification service (APNs)

Notification Infrastructure as a Service (NlaaS) – knock, Novu, Suprsend
Customer Engagement Platforms (CEPs) – Braze, Airship

- While platforms like **Novu** (open-source flexibility) and **Knock** (robust API management) offer strong foundations, there remains space for:
 - AI-based prediction + Intelligent prioritization
 - Multi-layer backup architecture
 - Administrative queue control
 - observability

Problem Statement

Design a framework that modern organizations require such that it is highly reliable, intelligent and a compliant notification framework that:

1. Delivers notifications across multiple channels based on user preference, priority, availability, and cost.
2. Prevents notification fatigue through batching, throttling, intelligent prioritization.
3. Ensures reliability using retries, exponential backoff, fallback mechanisms, dead-letter queues, and backup queue systems.
4. Provides observability and administrative control, including:
 - Delivery tracking (Sent, Delivered, Read, Failed)
 - Latency dashboards
 - Queue monitoring, pause/resume/terminate capabilities
5. Respects user privacy and preferences
6. Handles scalability constraints



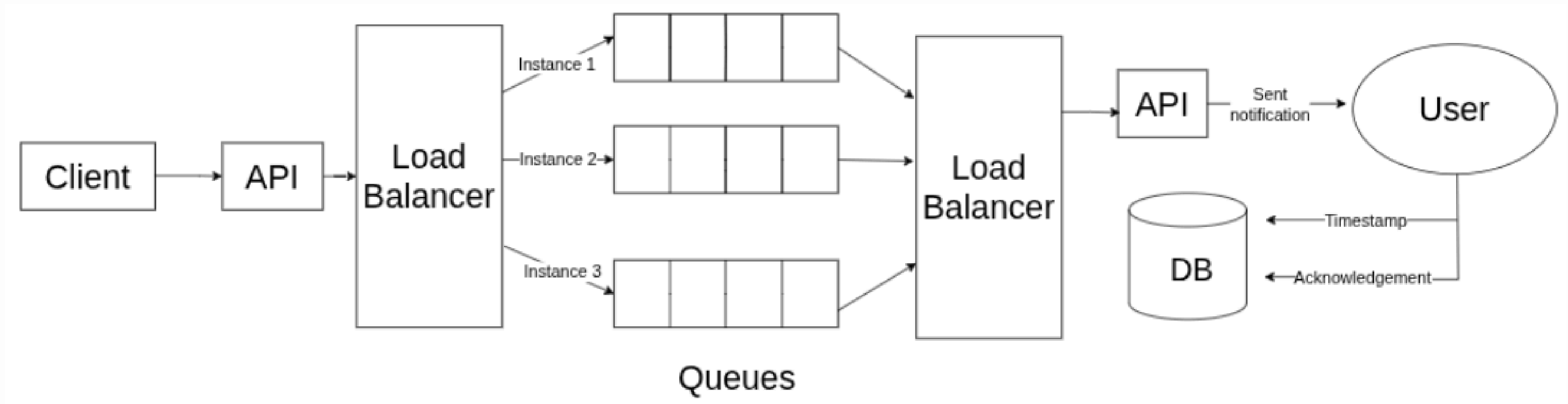
Our Understanding

From our understanding, this project is about building a centralized and scalable notification framework rather than just sending messages.

The objective is to design a system that reliably handles the complete notification lifecycle while respecting user preferences and ensuring fault tolerance. Whenever an event happens in an application (like a transaction, reminder, or alert), the system should: Check the user's preferences → Choose the correct channel → Send the notification → Handle the failures.

The scope focuses on building the notification infrastructure layer that can be integrated into applications. **The primary users** of this framework would be developers and system administrators who need a reliable and extensible notification solution. The system should be modular, configurable, and capable of handling increasing numbers of users and notifications efficiently.

Architecture Diagram



Objectives

Reliability: Ensuring message delivery through automated retry policies and dead-letter handling.

User Flexibility: Implementing a preference store where users manage notification types and preferred channels.

Scalability: Handling high-volume traffic through batching, scheduling, and throttling.

Rate Limiting: Prevent notification spam by controlling delivery frequency per user and system-wide.

Fault-tolerance: Ensure reliable delivery by handling failures without losing notifications.

Extensibility: Enable easy addition of new channels and features with minimal system changes.

Vital Features

1. **Omni Channel Support** (Push, Email, SMS, Pull, WhatsApp)– Deliver notifications through multiple channels based on priority, preference, and availability.

Reliability + reach + cost optimization.

Eg:

Event: PAYMENT_SUCCESS

- Push → Immediate alert
- Email → Detailed receipt
- In-app → Stored history

2. **Unified notification API** – Without a single API endpoint, when we add a new channel everytime we will have the need to create a new API . This leads to tight coupling, code duplication and hard maintenance. Instead of multiple API's we create a single entry point to send notifications irrespective of channels according to the user preferences.

Vital Features

3. **User Preferences** – Users must be able to control:

- Notification permission through which channels (for specific event)
- Do-not-disturb window
- Notification permission for which event types

4. **Scalability** – Scalability ensures the system can grow seamlessly to handle increasing demand without compromising performance or reliability.

5. **Rate limiting / throttling** – Includes (Limits are defined by hardware architecture)

for example:

- Per-user limits : max 5 notifications/min
- Global limits: max 1000 notifications/min

Vital Features

6. **Retry with exponential backoff** – Failed notification deliveries are retried with increasing delayed intervals. Supports:

- Retry on failure
- Exponential Backoff
- Max retry count

7. **Intelligent Prioritization** – Classifies notifications based on priority:
Classify:

- OTP → Immediate
- Promotion → Digest
- Reminder → Scheduled

8. **Dead letter handling** – Notifications that fail after the max retry count are moved to a dead letter queue for inspection.

Vital Features

9. **Monitoring logs** – Should store:

- Sent status
- Failed status
- Channel used
- Timestamp
- Error message

10. **Batching/Digest** – Combine multiple notifications.

Instead of:

- Assignment reminder
- Fee reminder
- Exam alert

Send: "You have 3 pending updates."

Vital Features

11. **Latency Dashboards** – Track system performance and usage.

Eg:

- 98% delivery rate
- 5% SMS failure rate
- 10% success rate
- Delivery latency 0.05 sec
- Client A used 80% monthly quota

Vital Features

12. **Send-time prediction** – Based on user activity patterns system should learn when a user typically opens app/ sends acknowledgements and should send notifications at that time when they are most active.

13. **Registration Token (API Key Management)** –

- Monthly quota limit
- Usage tracking

Eg:

- Client LMS → 100,000 notifications/month
- If exceeded → reject requests.

14. **Word / Character Limits** – Enforce per-channel limits after template rendering.

Eg: SMS > 160 chars → truncate or split.

Vital Features

15. Handling Large Docs & Videos (Using Links) – To keep the framework fast and secure, never send raw video or doc files directly. Instead secure, time-sensitive URLs that expire after use.

Eg: “Download your report here: <https://link...>”

16. Notification Tracking – Track lifecycle status.

- Sent
- Delivered
- Read
- Failed
- Marked as spam

Eg:

Email open → mark as “read”

SMS bounce → mark as “failed”

Vital Features

17. Email Signature – Every outgoing email automatically includes a standardized signature and an unsubscribe link to ensure branding consistency and allow users to opt out of future emails.

Eg: Email footer –

“XYZ Institute Notification System
support@xyz.edu

To unsubscribe from these emails, click here:

<https://xyz.edu/unsubscribe?token=abc123>”

18. Queue Backup Mechanism – A failover mechanism that switches processing to a secondary queue if the primary message queue becomes unavailable or stuck.

Eg: If primary_queue stops processing due to failure:

- Unprocessed jobs are redirected to backup_queue
- Workers resume processing from backup

Vital Features

19. **Queue Administrative Control** – Enables administrators to monitor queue states and pause, resume, or terminate specific queues during bottlenecks or abnormal processing behavior.

Eg: If a specific queue shows an abnormal failure, admin pauses or terminates it to prevent mass failures.

20. **Asynchronous Processing (Queue-Based)** – Notifications must NOT be sent directly from the API.

API → Queue → Worker → Send Notification

Advantages:

- Prevent API blocking
- Handle load
- Enable retries

Vital Features

21. **Webhook Services** – To receive real-time delivery updates from notification providers and keep our notification status updated accordingly.

- Delivery receipts
- Read receipts
- Bounce notifications
- Failure callbacks

Example: Email sent via provider –

- If delivered → mark as “Delivered”
- If opened → mark as “Read”
- If bounced → mark as “Failed”
- If failure → trigger fallback channel

Essential Feature

Idempotency Key Per Request – An idempotency key ensures that: If the same API request is sent multiple times (intentionally or accidentally), the system processes it only once. It prevents duplicate notifications.

Desired Feature

Security Scope –

Implement –

- API token auth
- Scoped event types per client
- HTTPS assumption

Technology Stack

Frontend: React 

MiddleWare: Node.js 

Backend: Python 

Async (Background worker system): Celery  Celery

Message Queue: RabbitMQ 

DataBase: MongoDB 

Notification Providers: SendGrid(Email), SendFire (SMS), UltraMsg(WhatsApp),
Firebase(Push).  Firebase

Tech-stack justification

React (Frontend)

Used to build a responsive admin dashboard for monitoring logs, queue status, delivery metrics with real-time updates.

Node.js (Middleware)

Handles the unified API with high concurrency support. Manages authentication, rate limiting, and routing.

Python (Backend)

Implements core logic, AI-based prioritization, send-time prediction, and regulatory validations.

Celery (Async Workers)

Processes background tasks like retries, batching, scheduling, and dead-letter handling to keep API latency low.



Tech-stack justification

RabbitMQ (Message Queue)

Ensures reliable, decoupled processing with priority queues, DLQ, and backup queue support.

MongoDB (Database)

Stores user preferences, notification logs, lifecycle states, and supports high write scalability.

Notification Providers

SendGrid (Email), SendFire (SMS), UltraMsg (WhatsApp), Firebase (Push) — enabling multi-channel delivery, fallback, redundancy, and compliance.





Features for Sprint 1

- Unified notification API - Poojitha, Rashmika
- User Preference (per channel, per event types) - Ruchika, Pranathi, Vandana
- Asynchronous Queue handling - Harshini, Mahani

BackUp Feature (To be planned)

- Notification Tracking



The image features a white background with decorative blue geometric shapes in the corners. A solid blue rectangle is in the top right corner. A solid blue rectangle is in the bottom right corner. In the bottom left corner, there are two overlapping light blue curved shapes. The text "THANK YOU!" is centered in a dark blue, bold, serif font.

THANK YOU!